



Department *of*
Software Engineering
BAHRIA UNIVERSITY
Discovering Knowledge

Lecture 3

Sorting Algorithms and exercise

Example 1: Maximum element

ALGORITHM *MaxElement*($A[0..n - 1]$)

//Determines the value of the largest element in a given array

//Input: An array $A[0..n - 1]$ of real numbers

//Output: The value of the largest element in A

$maxval \leftarrow A[0]$

for $i \leftarrow 1$ **to** $n - 1$ **do**

if $A[i] > maxval$

$maxval \leftarrow A[i]$

return $maxval$

Example 2: Element uniqueness problem

ALGORITHM *UniqueElements*($A[0..n - 1]$)

//Determines whether all the elements in a given array are distinct

//Input: An array $A[0..n - 1]$

//Output: Returns “true” if all the elements in A are distinct

// and “false” otherwise

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[i] = A[j]$ **return false**

return true

Example 3: Matrix multiplication

ALGORITHM *MatrixMultiplication*($A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$)

//Multiplies two n -by- n matrices by the definition-based algorithm

//Input: Two n -by- n matrices A and B

//Output: Matrix $C = AB$

for $i \leftarrow 0$ **to** $n - 1$ **do**

for $j \leftarrow 0$ **to** $n - 1$ **do**

$C[i, j] \leftarrow 0.0$

for $k \leftarrow 0$ **to** $n - 1$ **do**

$C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$

return C

Example 5: Counting binary digits

ALGORITHM *Binary*(n)

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

$count \leftarrow 1$

while $n > 1$ **do**

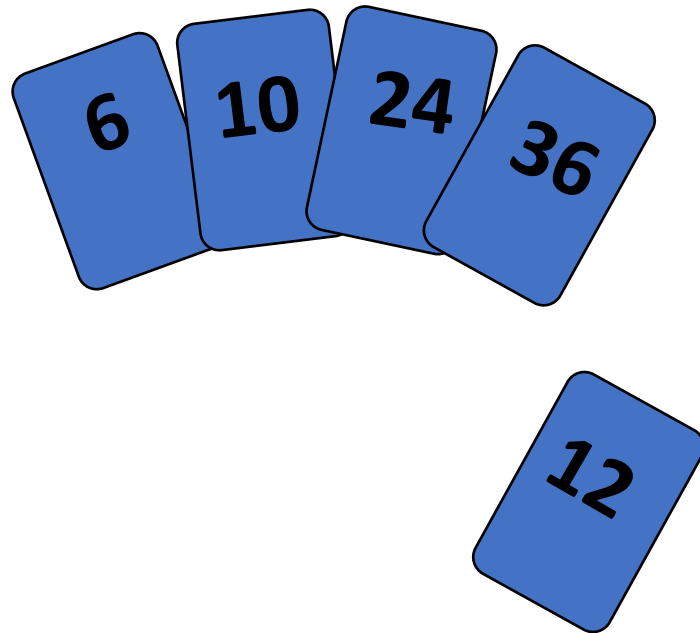
$count \leftarrow count + 1$

$n \leftarrow \lfloor n/2 \rfloor$

return $count$

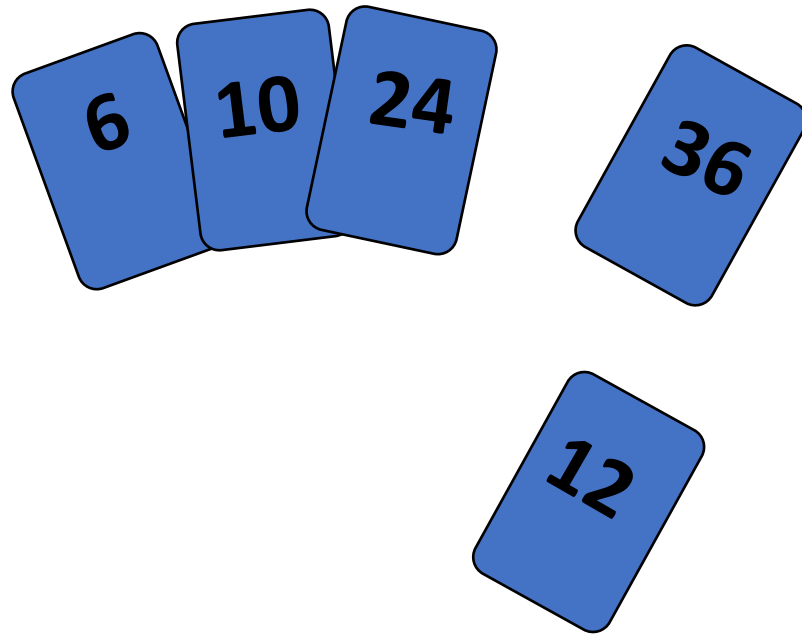
It cannot be investigated the way the previous examples are.

Insertion Sort

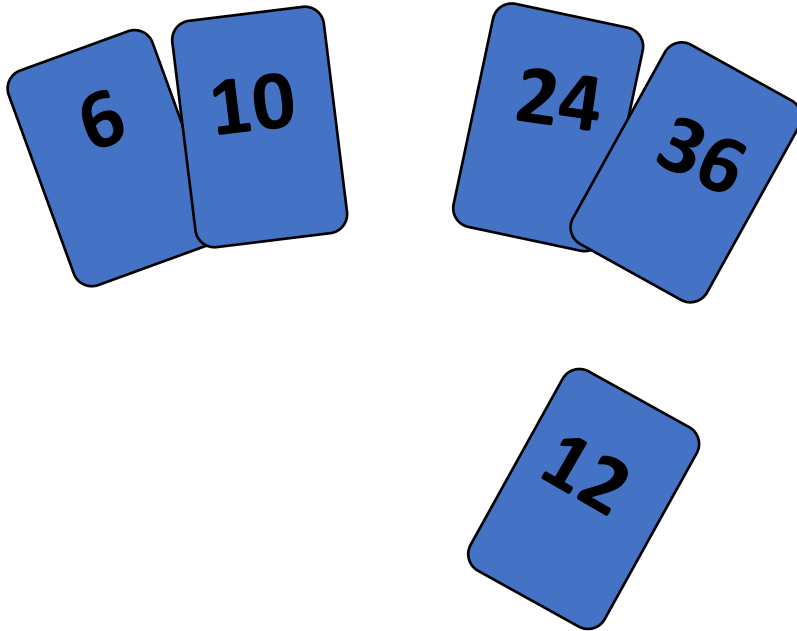


To insert 12, we need to make room for it by moving first 36 and then 24.

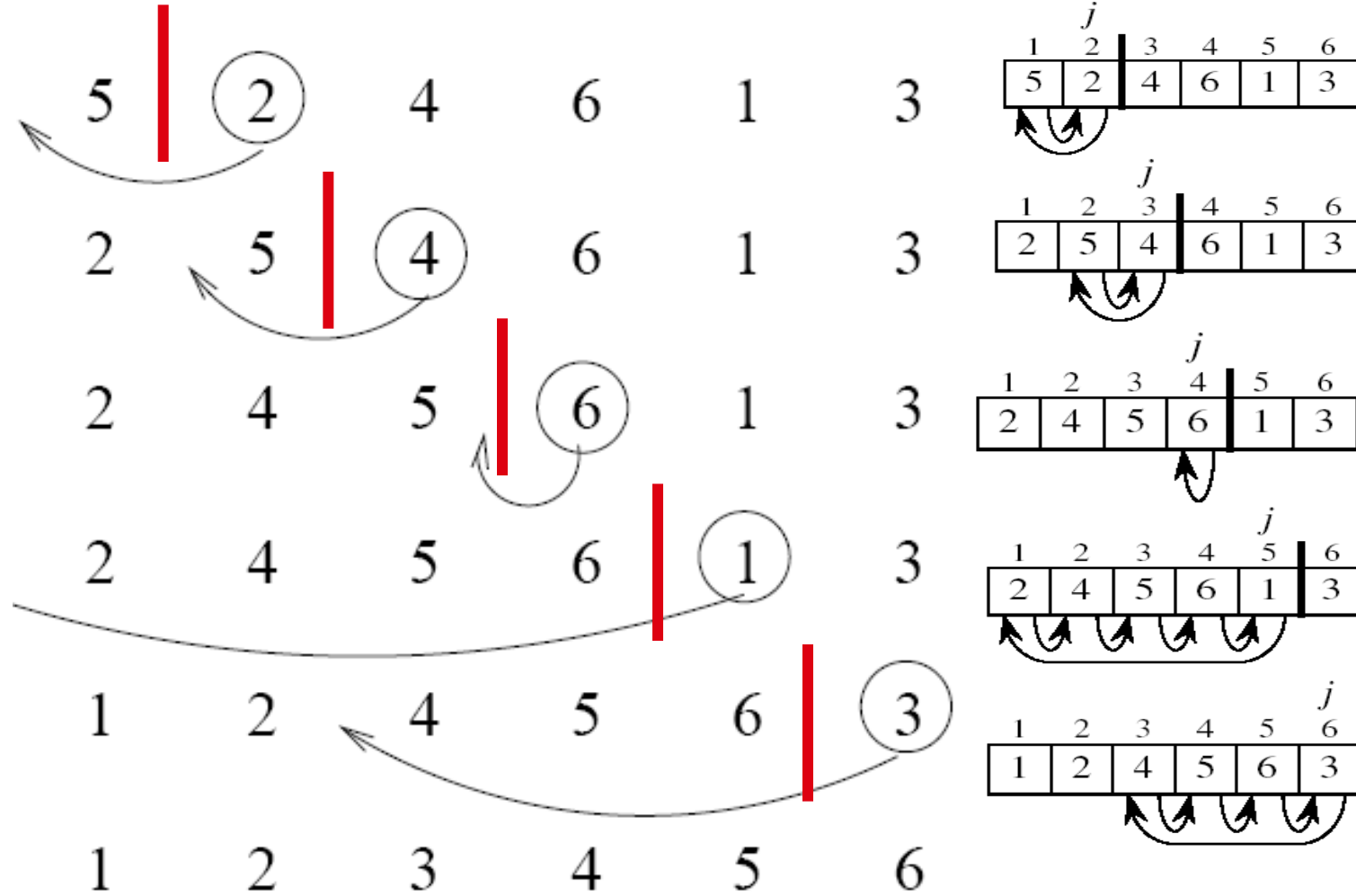
Insertion Sort



Insertion Sort



Insertion Sort



Analysis of Insertion Sort

INSERTION-SORT(A)

for $j \leftarrow 2$ **to** n

do $\text{key} \leftarrow A[j]$

 ▷ Insert $A[j]$ into the sorted sequence $A[1 \dots j-1]$

$i \leftarrow j - 1$

while $i > 0$ and $A[i] > \text{key}$

do $A[i + 1] \leftarrow A[i]$

$i \leftarrow i - 1$

$A[i + 1] \leftarrow \text{key}$

cost times

c_1 n

c_2 $n-1$

0 $n-1$

c_4 $n-1$

c_5 $\sum_{j=2}^n t_j$

c_6 $\sum_{j=2}^n (t_j - 1)$

c_7 $\sum_{j=2}^n (t_j - 1)$

c_8 $n-1$

t_j : # of times the while statement is executed at iteration j

$$T(n) = c_1 n + c_2 (n-1) + c_4 (n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8 (n-1)$$

Best Case Analysis

- The array is already sorted “while $i > 0$ and $A[i] > \text{key}$ ”

- $A[i] \leq \text{key}$ upon the first time the **while** loop test is run

(when $i = j - 1$)

- $t_j = 1$

- $T(n) = c_1n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_8(n - 1) =$

$$(c_1 + c_2 + c_4 + c_5 + c_8)n + (c_2 + c_4 + c_5 + c_8)$$

$$= an + b = \Theta(n)$$

$$T(n) = c_1n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n - 1)$$

Worst Case Analysis

- The array is in reverse sorted order “while i > 0 and A[i] > key”
 - Always $A[i] > \text{key}$ in **while** loop test
 - Have to compare **key** with all elements to the left of the j-th position
 $\Rightarrow$ compare with j-1 elements $\Rightarrow t_j = j$

using $\sum_{j=1}^n j = \frac{n(n+1)}{2} \Rightarrow \sum_{j=2}^n j = \frac{n(n+1)}{2} - 1 \Rightarrow \sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$ we have:

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) + c_6 \frac{n(n-1)}{2} + c_7 \frac{n(n-1)}{2} + c_8(n-1)$$

$$= an^2 + bn + c \quad \text{a quadratic function of } n$$

- $T(n) = \Theta(n^2)$ order of growth in n^2

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1)$$

Comparisons and Exchanges in Insertion Sort

INSERTION-SORT(A)

for $j \leftarrow 2$ **to** n

cost times

c_1 n

do $\text{key} \leftarrow A[j]$

c_2 $n-1$

Insert $A[j]$ into the sorted sequence $A[1 \dots j-1]$

0 $n-1$

$i \leftarrow j - 1$

$\approx n^2/2$ comparisons

c_4 $n-1$

while $i > 0$ and $A[i] > \text{key}$

c_5 $\sum_{j=2}^n t_j$

do $A[i + 1] \leftarrow A[i]$

c_6 $\sum_{j=2}^n (t_j - 1)$

$i \leftarrow i - 1$

$\approx n^2/2$ exchanges

c_7 $\sum_{j=2}^n (t_j - 1)$

$A[i + 1] \leftarrow \text{key}$

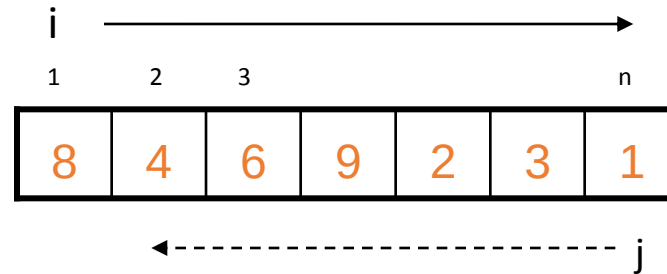
c_8 $n-1$

Insertion Sort - Summary

- Advantages
 - Good running time for “almost sorted” arrays $\Theta(n)$
- Disadvantages
 - $\Theta(n^2)$ running time in **worst** and **average** case
 - $\approx n^2/2$ **comparisons** and **exchanges**

Bubble Sort

- Idea:
 - Repeatedly pass through the array
 - Swaps adjacent elements that are out of order



- Easier to implement, but slower than Insertion sort

Example

8	4	6	9	2	3	1
---	---	---	---	---	---	---

$i = 1$ ←----- j

8	4	6	9	2	1	3
---	---	---	---	---	---	---

$i = 1$ ←----- j

8	4	6	9	1	2	3
---	---	---	---	---	---	---

$i = 1$ ←----- j

8	4	6	1	9	2	3
---	---	---	---	---	---	---

$i = 1$ ←----- j

8	4	1	6	9	2	3
---	---	---	---	---	---	---

$i = 1$ ←----- j

8	1	4	6	9	2	3
---	---	---	---	---	---	---

$i = 1$ j

1	8	4	6	9	2	3
---	---	---	---	---	---	---

$i = 1$ j

1	8	4	6	9	2	3
---	---	---	---	---	---	---

$i = 2$ j

1	2	8	4	6	9	3
---	---	---	---	---	---	---

$i = 3$ j

1	2	3	8	4	6	9
---	---	---	---	---	---	---

$i = 4$ j

1	2	3	4	8	6	9
---	---	---	---	---	---	---

$i = 5$ j

1	2	3	4	6	8	9
---	---	---	---	---	---	---

$i = 6$ j

1	2	3	4	6	8	9
---	---	---	---	---	---	---

$i = 7$

j

Bubble Sort

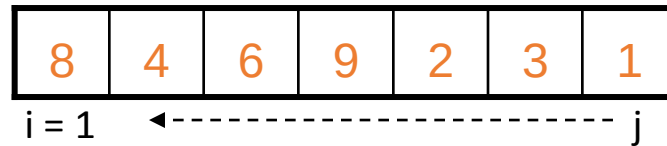
Alg.: BUBBLESORT(A)

for $i \leftarrow 1$ to $\text{length}[A]$

do for $j \leftarrow \text{length}[A]$ downto $i + 1$

do if $A[j] < A[j - 1]$

then exchange $A[j] \leftrightarrow A[j - 1]$



Bubble-Sort Running Time

Alg.: BUBBLESORT(A)

for $i \leftarrow 1$ to $\text{length}[A]$ c_1

do for $j \leftarrow \text{length}[A]$ downto $i + 1$ c_2

Comparisons: $\approx n^2/2$ do if $A[j] < A[j - 1]$ c_3

Exchanges: $\approx n^2/2$ then exchange $A[j] \leftrightarrow A[j - 1]$ c_4

$$T(n) = c_1(n+1) + c_2 \sum_{i=1}^n (n-i+1) + c_3 \sum_{i=1}^n (n-i) + c_4 \sum_{i=1}^n (n-i)$$

$$= \Theta(n) + (c_2 + c_3 + c_4) \sum_{i=1}^n (n-i)$$

$$\text{where } \sum_{i=1}^n (n-i) = \sum_{i=1}^n n - \sum_{i=1}^n i = n^2 - \frac{n(n+1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

$$\text{Thus, } T(n) = \Theta(n^2)$$

Complexity of Bubble Sort

- Time consuming operations
 - compares, swaps.
- Compares
 - a *for* loop embedded inside a *for* loop
 - $(n-1)+(n-2)+(n-3) \dots +1$, or $O(n^2)$
- Swaps
 - inside a conditional -> #swaps data dependent !!
 - Best Case 0, or $O(1)$
 - Worst Case $(n-1)+(n-2)+(n-3) \dots +1$, or $O(n^2)$
 - Average Case $O(n^2)$
- Space
 - size of the array
 - an *in-place* algorithm

Pass	#comparisons
1	$n - 1$
2	$n - 2$
3	$n - 3$
...	...
$n - 1$	1

$$\sum_{i=1}^{n-1} i = \frac{(n-1)(n)}{2} = \frac{n^2 - n}{2} \cong n^2$$

Selection Sort

- Idea:
 - Find the smallest element in the array
 - Exchange it with the element in the first position
 - Find the second smallest element and exchange it with the element in the second position
 - Continue until the array is sorted
- Disadvantage:
 - Running time depends only slightly on the amount of order in the file

Example

8	4	6	9	2	3	1
---	---	---	---	---	---	---

1	4	6	9	2	3	8
---	---	---	---	---	---	---

1	2	6	9	4	3	8
---	---	---	---	---	---	---

1	2	3	9	4	6	8
---	---	---	---	---	---	---

1	2	3	4	9	6	8
---	---	---	---	---	---	---

1	2	3	4	6	9	8
---	---	---	---	---	---	---

1	2	3	4	6	8	9
---	---	---	---	---	---	---

1	2	3	4	6	8	9
---	---	---	---	---	---	---

Selection Sort

Alg.: SELECTION-SORT(A)

$n \leftarrow \text{length}[A]$

for $j \leftarrow 1$ to $n - 1$

 do $\text{smallest} \leftarrow j$

 for $i \leftarrow j + 1$ to n

 do if $A[i] < A[\text{smallest}]$

 then $\text{smallest} \leftarrow i$

 exchange $A[j] \leftrightarrow A[\text{smallest}]$

8	4	6	9	2	3	1
---	---	---	---	---	---	---

Analysis of Selection Sort

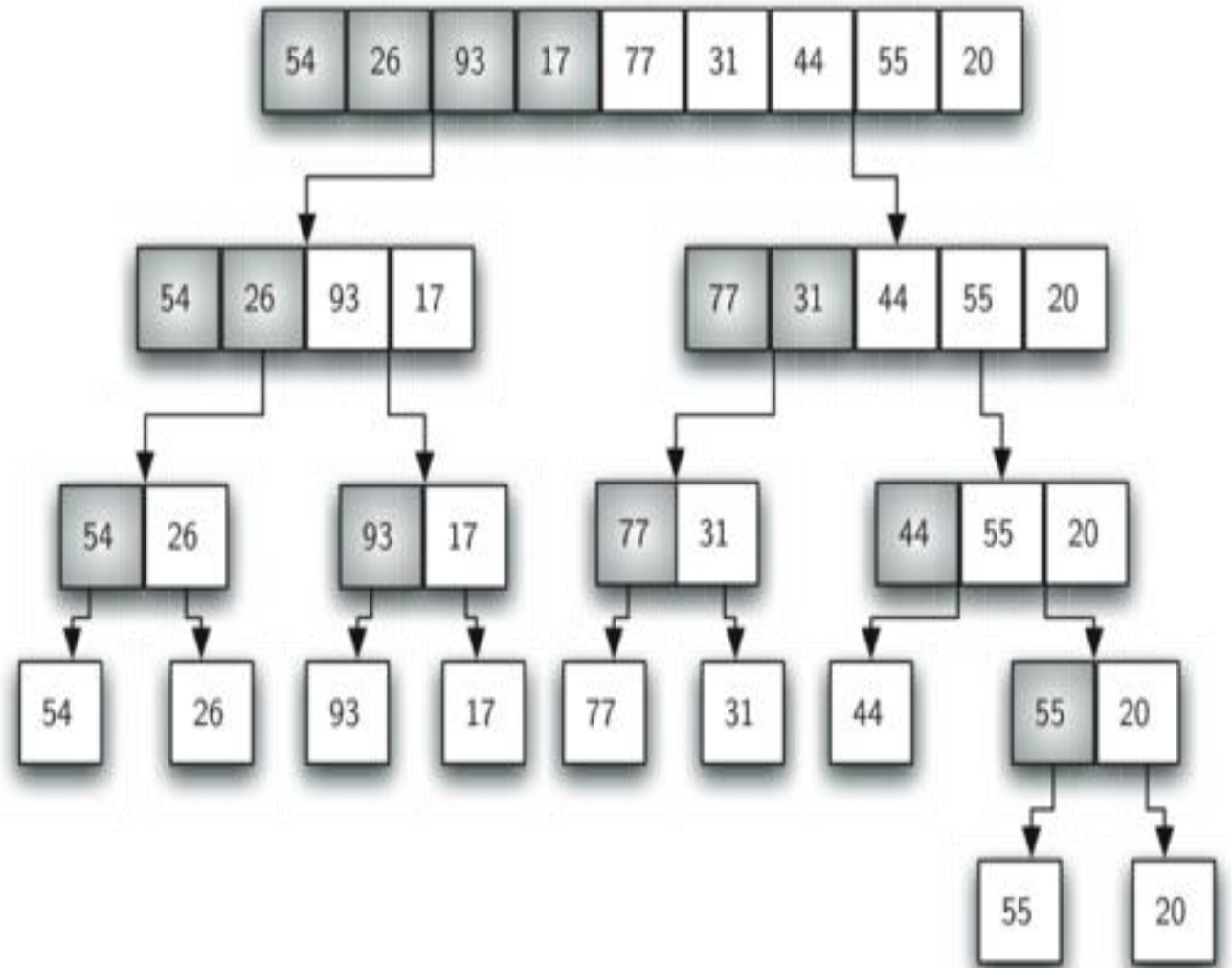
Alg.: SELECTION-SORT(A)

	cost	times
$n \leftarrow \text{length}[A]$	c_1	1
for $j \leftarrow 1$ to $n - 1$	c_2	n
do $\text{smallest} \leftarrow j$	c_3	$n-1$
$\approx n^2/2$ comparisons for $i \leftarrow j + 1$ to n	c_4	$\sum_{j=1}^{n-1} (n - j + 1)$
do if $A[i] < A[\text{smallest}]$	c_5	$\sum_{j=1}^{n-1} (n - j)$
then $\text{smallest} \leftarrow i$	c_6	$\sum_{j=1}^{n-1} (n - j)$
$\approx n$ exchanges exchange $A[j] \leftrightarrow A[\text{smallest}]$	c_7	$n-1$

$$T(n) = c_1 + c_2 n + c_3 (n-1) + c_4 \sum_{j=1}^{n-1} (n-j+1) + c_5 \sum_{j=1}^{n-1} (n-j) + c_6 \sum_{j=1}^{n-1} (n-j) + c_7 (n-1) = \Theta(n^2)$$

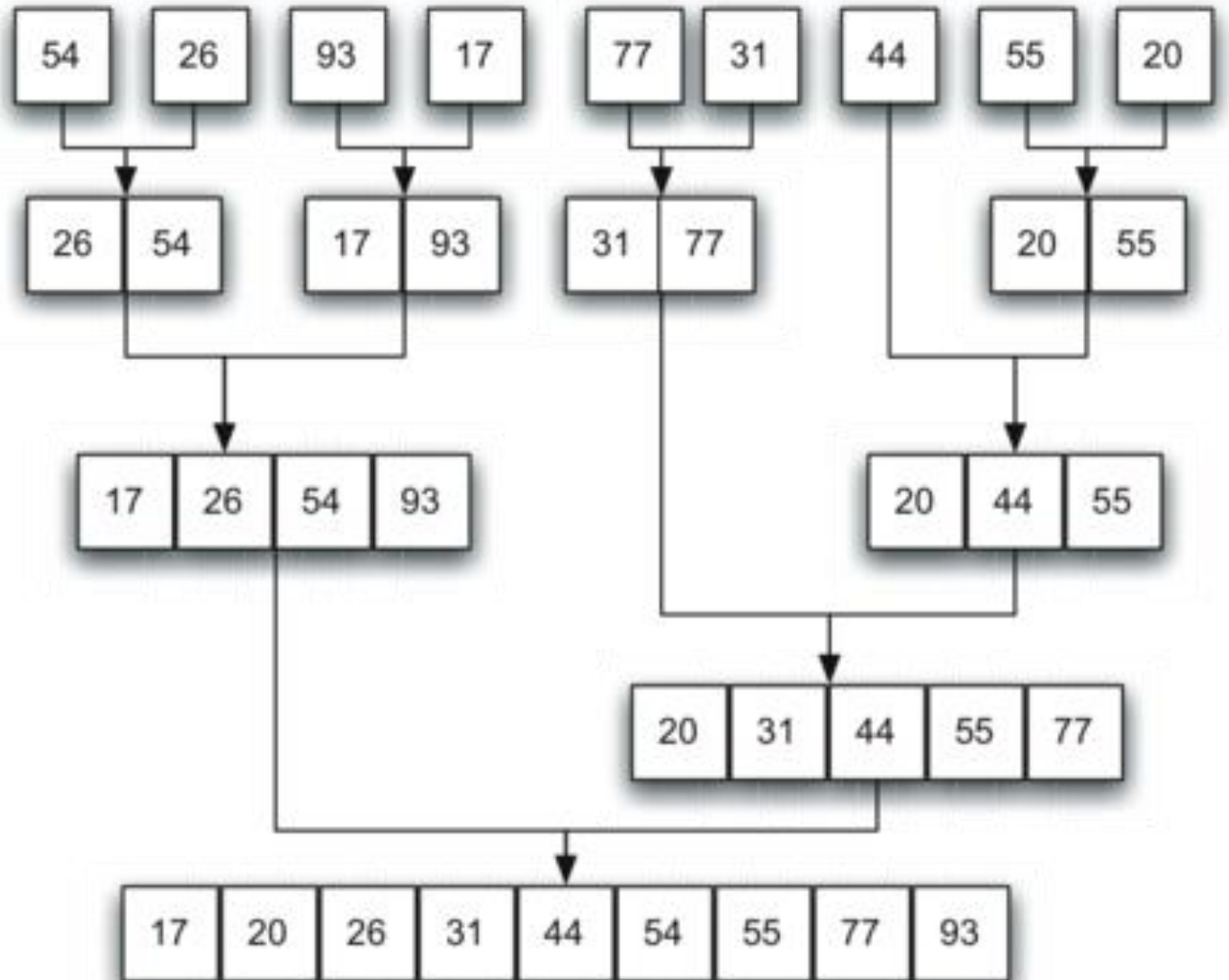
Mergesort

- Mergesort (divide-and-conquer)
 - Divide array into two halves.



Mergesort

- Mergesort (divide-and-conquer)
 - Divide array into two halves.
 - Recursively sort each half.
 - Merge two halves to make sorted whole.



Analysis of Merge Sort:

Let $T(n)$ be the total time taken by the Merge Sort algorithm.

- Sorting two halves will take at the most $2T\left(\frac{n}{2}\right)$ time.
- When we merge the sorted lists, we come up with a total $n-1$ comparison because the last element which is left will need to be copied down in the combined list, and there will be no comparison.

Thus, the relational formula will be

$$T(n) = 2T\left(\frac{n}{2}\right) + n - 1$$

$$\text{So } T(n) = 2T\left(\frac{n}{2}\right) + n \dots \text{equation 1}$$

Putting $n=\frac{n}{2}$ in place of n inequation 1

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \dots \text{equation 2}$$

Put 2 equation in 1 equation

$$T(n) = 2 \left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \right] + n$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + \frac{2n}{2} + n$$

$$T(n) = 2^2 T\left(\frac{n}{2^2}\right) + 2n \dots \text{equation 3}$$

Putting $n=\frac{n}{2^2}$ in equation 1

$$T\left(\frac{n}{2^2}\right) = 2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \dots \text{equation 4}$$

Putting 4 equation in 3 equation

$$T(n) = 2^2 \left[2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \right] + 2n$$

$$T(n) = 2^3 T\left(\frac{n}{2^3}\right) + n + 2n$$

$$T(n) = 2^3 T\left(\frac{n}{2^3}\right) + 3n \dots \text{equation 5}$$

From eq 1, eq3, eq 5.....we get

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + in \dots \dots \dots \text{equation 6}$$

From Stopping Condition:

$$\frac{n}{2^i} = 1 \text{ And } T\left(\frac{n}{2^i}\right) = 0$$

$$n = 2^i$$

Apply log both sides:

$$\log n = \log 2^i$$

$$\log n = i \log 2$$

$$\frac{\log n}{\log 2} = i$$

$$\log_2 n = i$$

From 6 equation

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + in$$

$$= 2^i \times 0 + \log_2 n \cdot n$$

$$= T(n) = n \cdot \log n$$

Best Case Complexity: The merge sort algorithm has a best-case time complexity of **$O(n \cdot \log n)$** for the already sorted array.

Average Case Complexity: The average-case time complexity for the merge sort algorithm is **$O(n \cdot \log n)$** , which happens when 2 or more elements are jumbled, i.e., neither in the ascending order nor in the descending order.

Worst Case Complexity: The worst-case time complexity is also **$O(n \cdot \log n)$** , which occurs when we sort the descending order of an array into the ascending order.

Mergesort recursive

```
int MergeSort(int A[],int lb,int ub)
{
    if lb > ub
        return
    int mid = (lb+ub)/2
    mergeSort(A, lb, mid)
    mergeSort(A, mid+1, ub)
    merge(A, lb, ub, mid)
}
```

Sorting Algorithms

- Quick Sort
 - The quick sort algorithm works by sub-dividing an array into two pieces and then calling itself recursively to sort the pieces
 - The following pseudo-code shows the algorithm at a high level:

Quick Sort

```
/* low --> Starting index, high --> Ending index */
```

```
quickSort(arr[], low, high)
```

```
{
```

```
    if (low < high)
```

```
    {
```

```
        /* pi is partitioning index, arr[pi] is now  
        at right place */
```

```
        pi = partition(arr, low, high);
```

```
        quickSort(arr, low, pi - 1); // Before pi
```

```
        quickSort(arr, pi + 1, high); // After pi
```

```
    }
```

```
}
```

```
/* This function takes last element as pivot, places the pivot element at its correct  
position in sorted array, and places all smaller (smaller than pivot) to left of pivot and  
all greater elements to right of pivot */
```

```
partition (arr[], low, high)
```

```
{
```

```
    // pivot (Element to be placed at right position)  
    pivot = arr[high];
```

```
    i = (low - 1) // Index of smaller element
```

```
    for (j = low; j <= high- 1; j++)
```

```
    {
```

```
        // If current element is smaller than the pivot  
        if (arr[j] < pivot)
```

```
    {
```

```
        i++; // increment index of smaller element  
        swap arr[i] and arr[j]
```

```
    }
```

```
    }
```

```
    swap arr[i + 1] and arr[high])
```

```
    return (i + 1)
```

```
}
```

Equation:

$$T(n) = T(1) + T(n-1) + n$$

T (1) is time taken by pivot element.

T (n-1) is time taken by remaining element except for pivot element.

N: the number of comparisons required to identify the exact position of itself (every element)

$$T(n) = (n-1) T(1) + T(n-(n-1)) + (n-(n-2)) + (n-(n-3)) + (n-(n-4)) + n$$

$$T(n) = (n-1) T(1) + T(1) + 2 + 3 + 4 + \dots + n$$

$$T(n) = (n-1) T(1) + T(1) + 2 + 3 + 4 + \dots + n + 1 - 1$$

[Adding 1 and subtracting 1 for making AP series]

$$T(n) = (n-1) T(1) + T(1) + 1 + 2 + 3 + 4 + \dots + n - 1$$

$$T(n) = (n-1) T(1) + T(1) + \frac{n(n+1)}{2} - 1$$

Stopping Condition: T (1) = 0

Because at last there is only one element left and no comparison is required.

$$T(n) = (n-1) (0) + 0 + \frac{n(n+1)}{2} - 1$$

$$T(n) = \frac{n^2 + n - 2}{2}$$

$$T(n) = O(n^2)$$

Avoid all the terms except higher terms n^2

Relational Formula for Worst Case:

$$T(n) = T(1) + T(n-1) + n \dots \dots \dots (1)$$

$$T(n-1) = T(1) + T(n-1-1) + (n-1)$$

Put T (n-1) in equation 1

By putting (n-1) in place of n in equation 1

$$T(n) = T(1) + T(1) + T(n-2) + (n-1) + n \dots \dots \dots (ii)$$

$$T(n) = 2T(1) + T(n-2) + (n-1) + n$$

$$T(n-2) = T(1) + T(n-3) + (n-2)$$

Put T (n-2) in equation (ii)

By putting (n-2) in place of n in equation 1

$$T(n) = 2T(1) + T(1) + T(n-3) + (n-2) + (n-1) + n$$

$$T(n) = 3T(1) + T(n-3) + (n-2) + (n-1) + n$$

$$T(n-3) = T(1) + T(n-4) + n-3$$

By putting (n-3) in place of n in equation 1

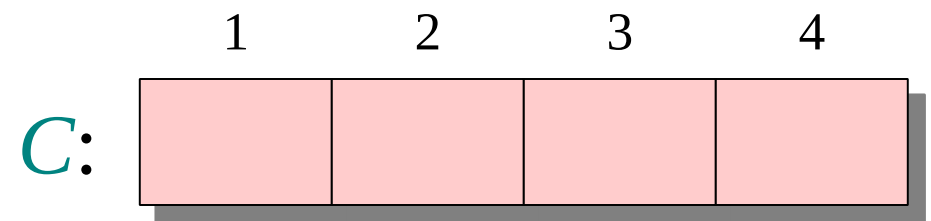
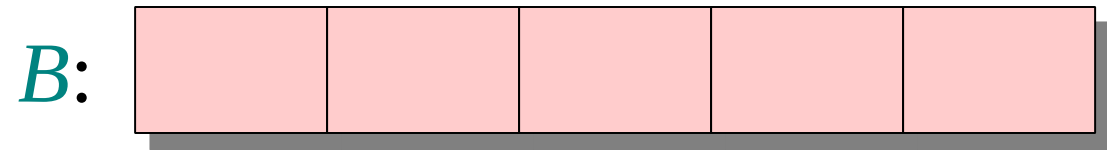
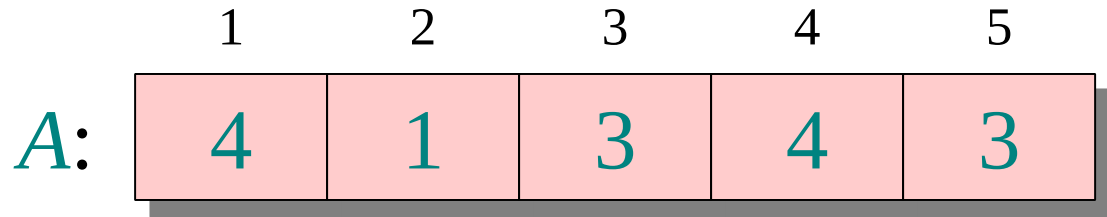
$$T(n) = 3T(1) + T(1) + T(n-4) + (n-3) + (n-2) + (n-1) + n$$

$$= 4T(1) + T(n-4) + (n-3) + (n-2) + (n-1) + n \dots \dots \dots (iii)$$

Counting sort

```
for  $i \leftarrow 1$  to  $k$   
  do  $C[i] \leftarrow 0$   
for  $j \leftarrow 1$  to  $n$   
  do  $C[A[j]] \leftarrow C[A[j]] + 1$   $\triangleright C[i] = |\{\text{key} = i\}|$   
for  $i \leftarrow 2$  to  $k$   
  do  $C[i] \leftarrow C[i] + C[i-1]$   $\triangleright C[i] = |\{\text{key} \leq i\}|$   
for  $j \leftarrow n$  downto  $1$   
  do  $B[C[A[j]]] \leftarrow A[j]$   
     $C[A[j]] \leftarrow C[A[j]] - 1$ 
```


Counting-sort example



Loop 1

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

<i>B</i> :					
------------	--	--	--	--	--

	1	2	3	4
<i>C</i> :	0	0	0	0

for $i \leftarrow 1$ **to** k
 do $C[i] \leftarrow 0$

Loop 2

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

<i>B</i> :					
------------	--	--	--	--	--

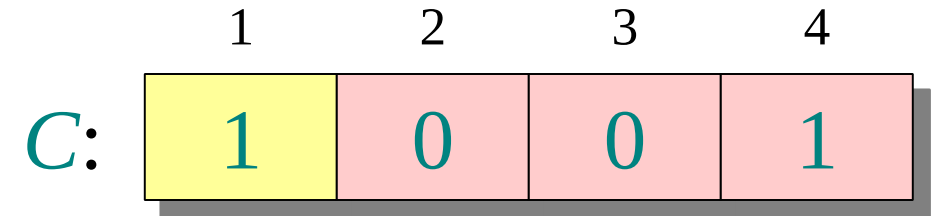
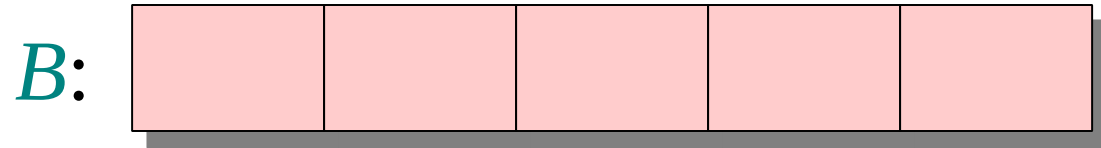
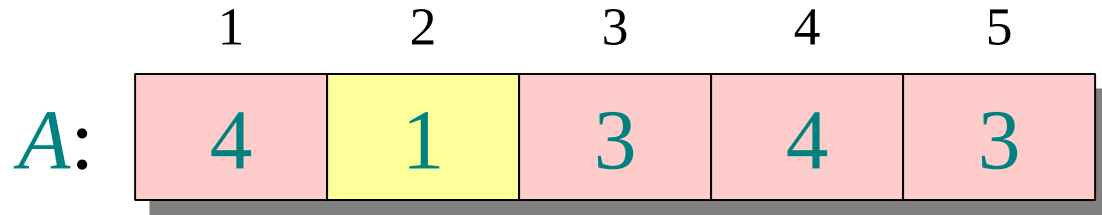
	1	2	3	4
<i>C</i> :	0	0	0	1

for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1$

$\triangleright C[i] = |\{\text{key} = i\}|$

Loop 2



for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1$

▷ $C[i] = |\{\text{key} = i\}|$

Loop 2

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

<i>B</i> :					
------------	--	--	--	--	--

	1	2	3	4
<i>C</i> :	1	0	1	1

for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1$

$\triangleright C[i] = |\{\text{key} = i\}|$

Loop 2

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

<i>B</i> :					
------------	--	--	--	--	--

	1	2	3	4
<i>C</i> :	1	0	1	2

for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1$

$\triangleright C[i] = |\{\text{key} = i\}|$

Loop 2

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

<i>B</i> :					
------------	--	--	--	--	--

	1	2	3	4
<i>C</i> :	1	0	2	2

for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1$

$\triangleright C[i] = |\{\text{key} = i\}|$

Loop 3

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

<i>B</i> :					
------------	--	--	--	--	--

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>C'</i> :	1	1	2	2
-------------	---	---	---	---

for $i \leftarrow 2$ **to** k

do $C[i] \leftarrow C[i] + C[i-1]$ $\triangleright C[i] = |\{\text{key} \leq i\}|$

Loop 3

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

<i>B</i> :					
------------	--	--	--	--	--

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>C'</i> :	1	1	3	2
-------------	---	---	---	---

for $i \leftarrow 2$ **to** k

do $C[i] \leftarrow C[i] + C[i-1]$ $\triangleright C[i] = |\{\text{key} \leq i\}|$

Loop 3

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

<i>B</i> :					
------------	--	--	--	--	--

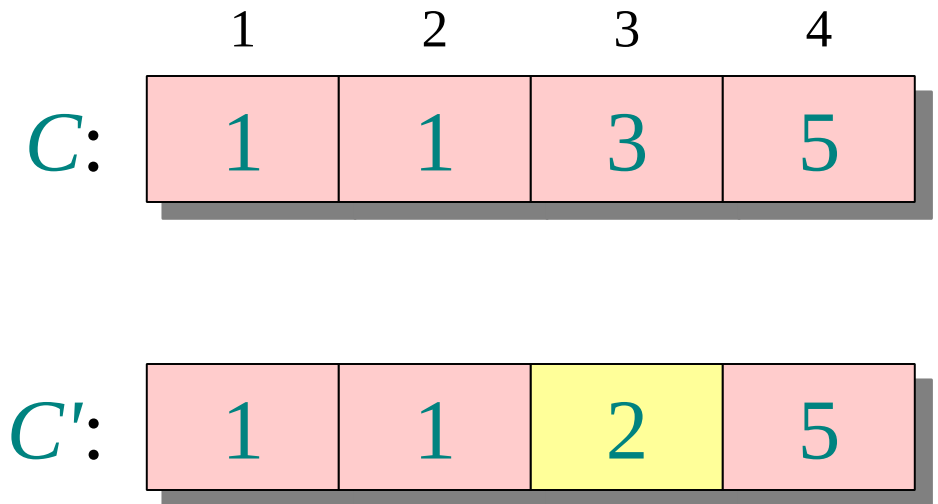
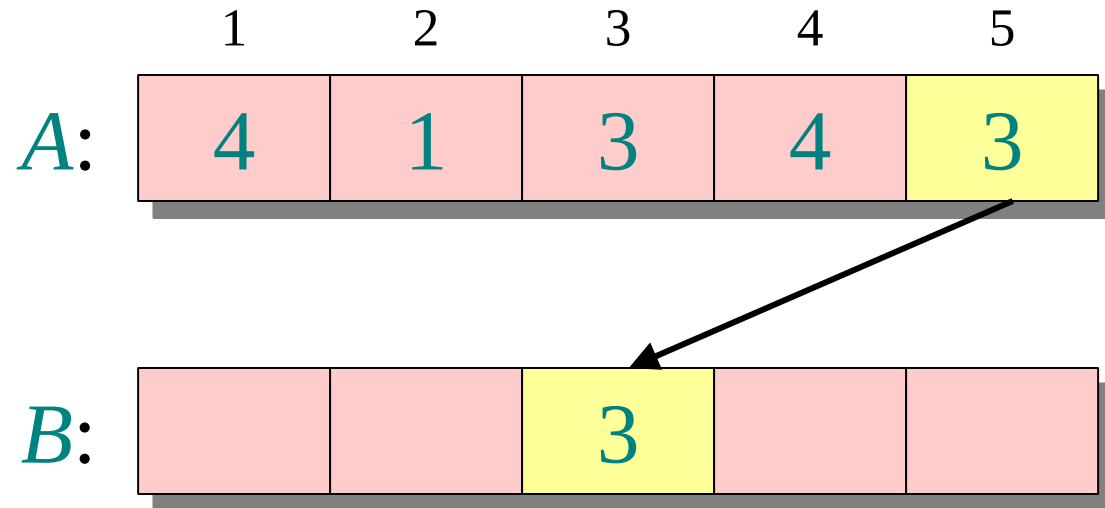
	1	2	3	4
<i>C</i> :	1	0	2	2

<i>C'</i> :	1	1	3	5
-------------	---	---	---	---

for $i \leftarrow 2$ **to** k

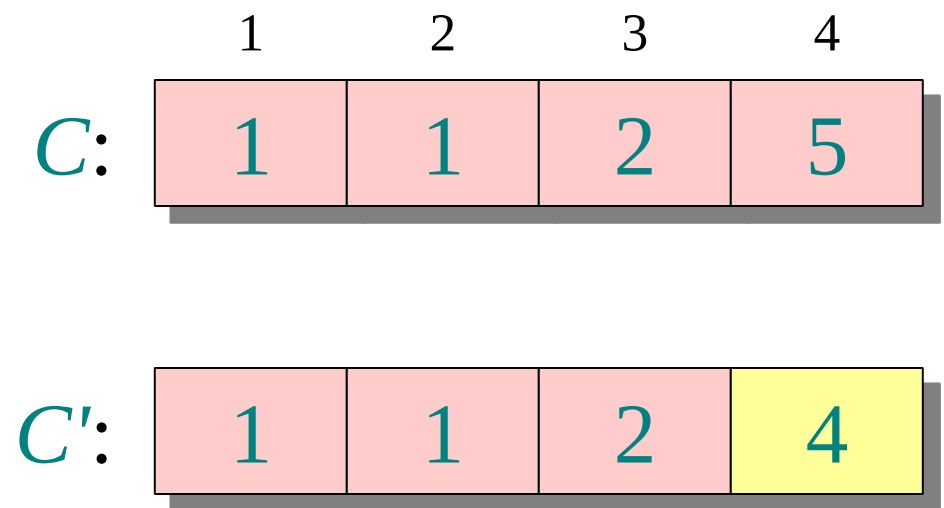
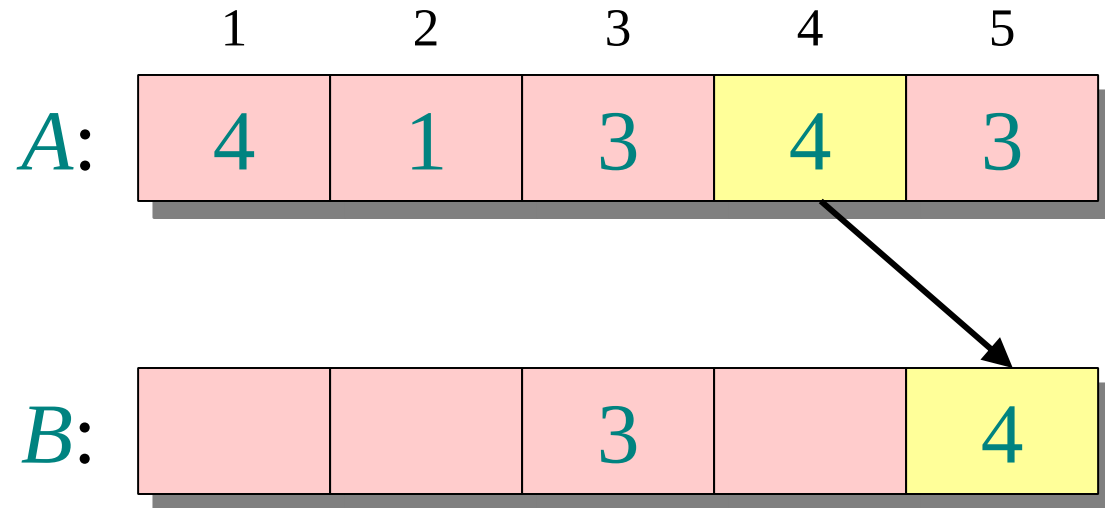
do $C[i] \leftarrow C[i] + C[i-1]$ $\triangleright C[i] = |\{\text{key} \leq i\}|$

Loop 4



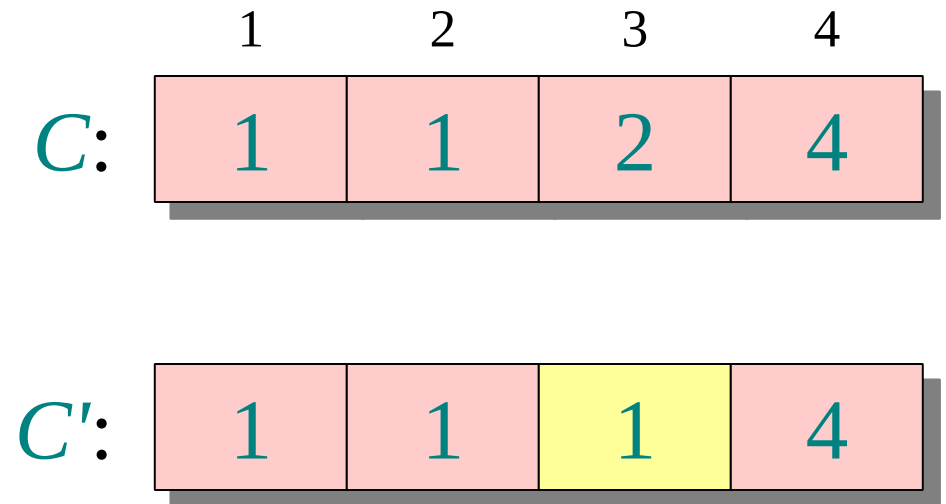
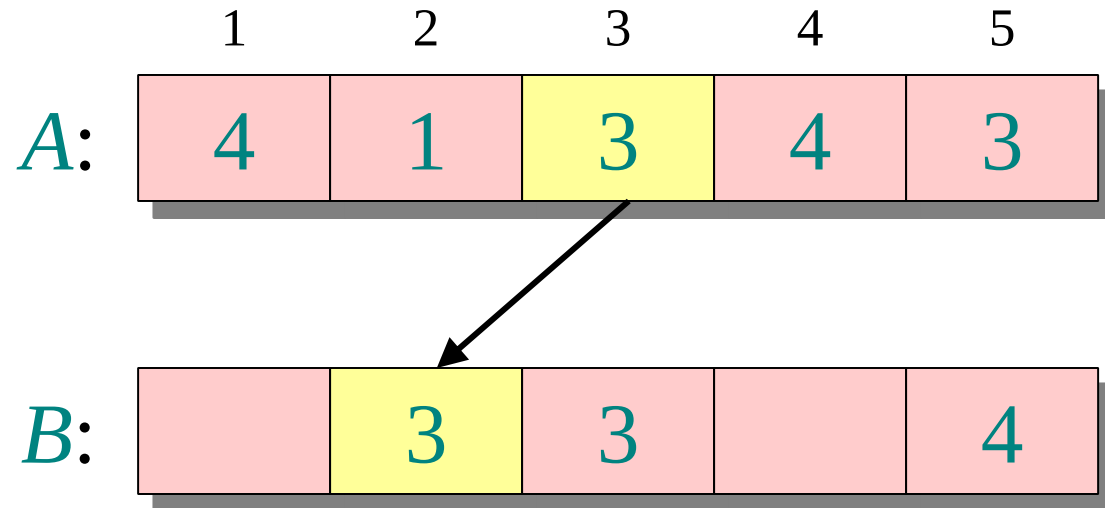
```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
       $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Loop 4



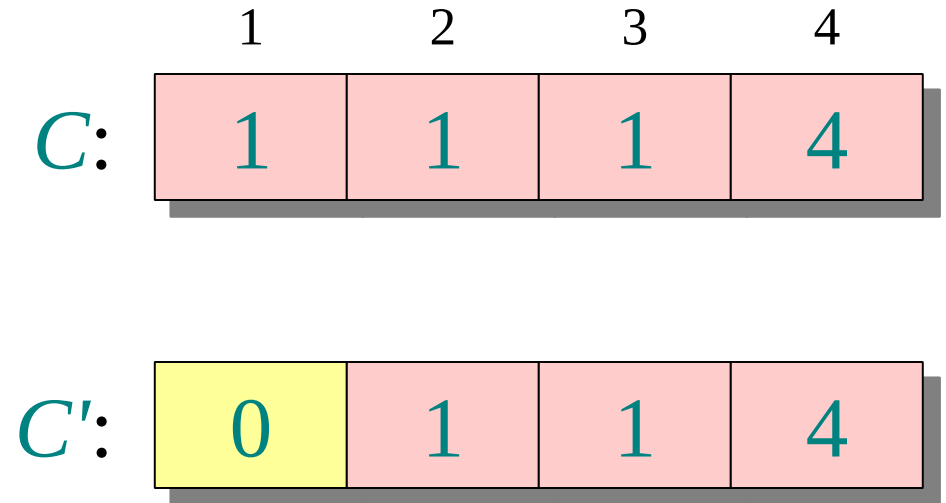
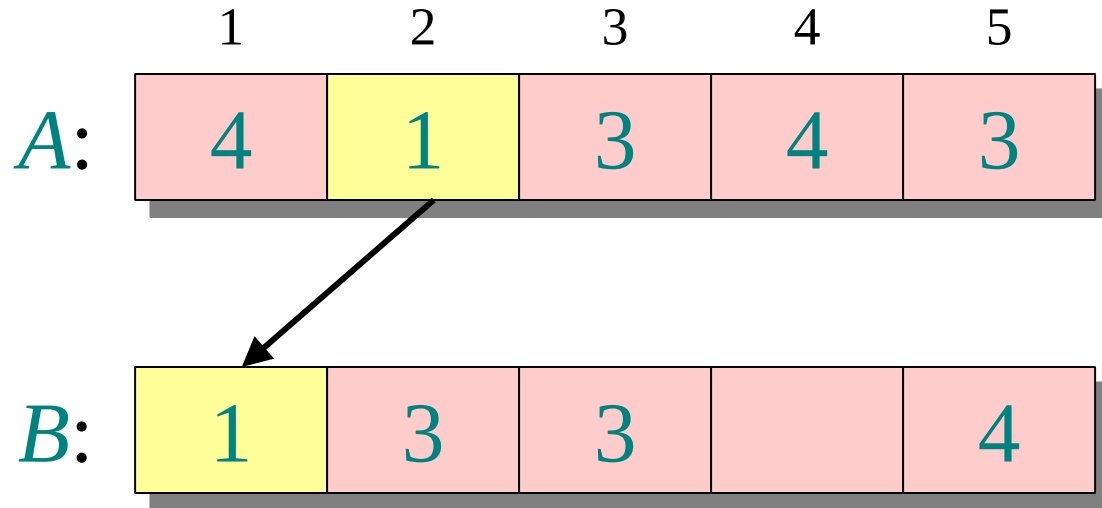
```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
       $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Loop 4



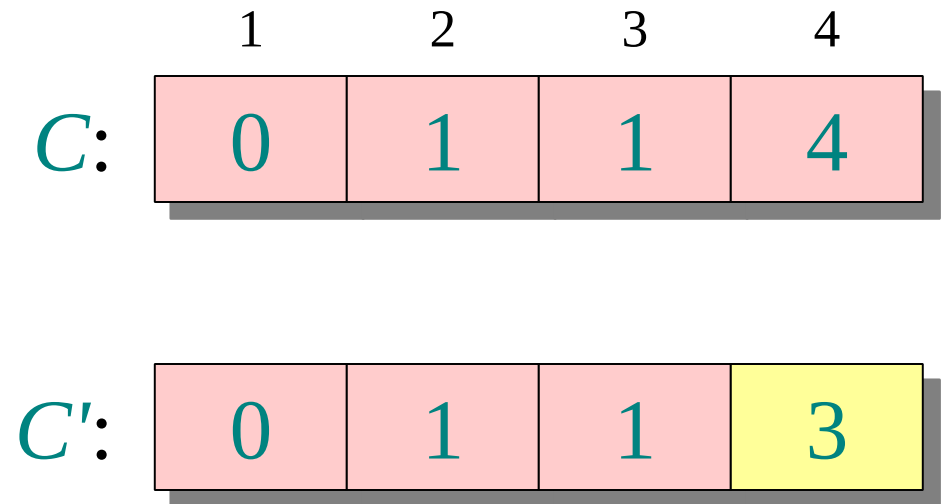
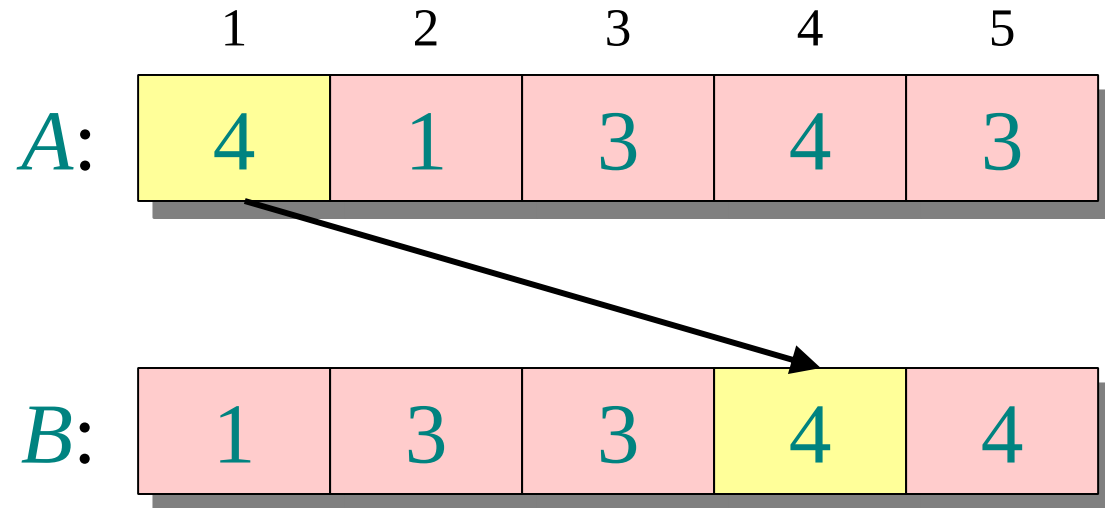
```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
       $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Loop 4



```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
       $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Loop 4



```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
       $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Analysis

$\Theta(k)$

for $i \leftarrow 1$ **to** k
 do $C[i] \leftarrow 0$

$\Theta(n)$

for $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1$

$\Theta(k)$

for $i \leftarrow 2$ **to** k
 do $C[i] \leftarrow C[i] + C[i-1]$

$\Theta(n)$

for $j \leftarrow n$ **downto** 1
 do $B[C[A[j]]] \leftarrow A[j]$
 $C[A[j]] \leftarrow C[A[j]] - 1$

$\Theta(n + k)$

Heap Sort

Adding 3, 5, 1, 19, 11, and 22 to a heap, initially empty

3

(a) After adding 3

5
3

(b) After adding 5

5
3 1

(c) After adding 1

19
5 1
3

(d) After adding 19

19
11 1
3 5

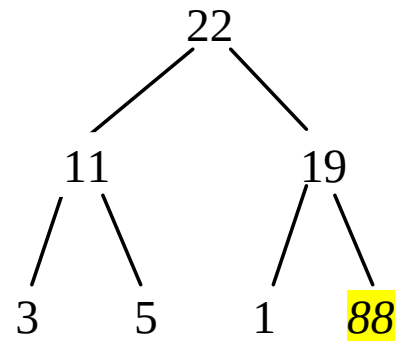
(e) After adding 11

22
11 19
3 5 1

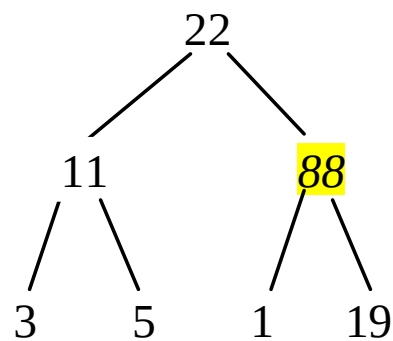
(f) After adding 22

Rebuild the heap after adding a new node

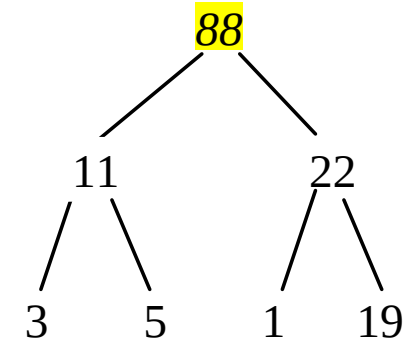
Adding 88 to the heap



(a) Add 88 to a heap



(b) After swapping 88 with 19

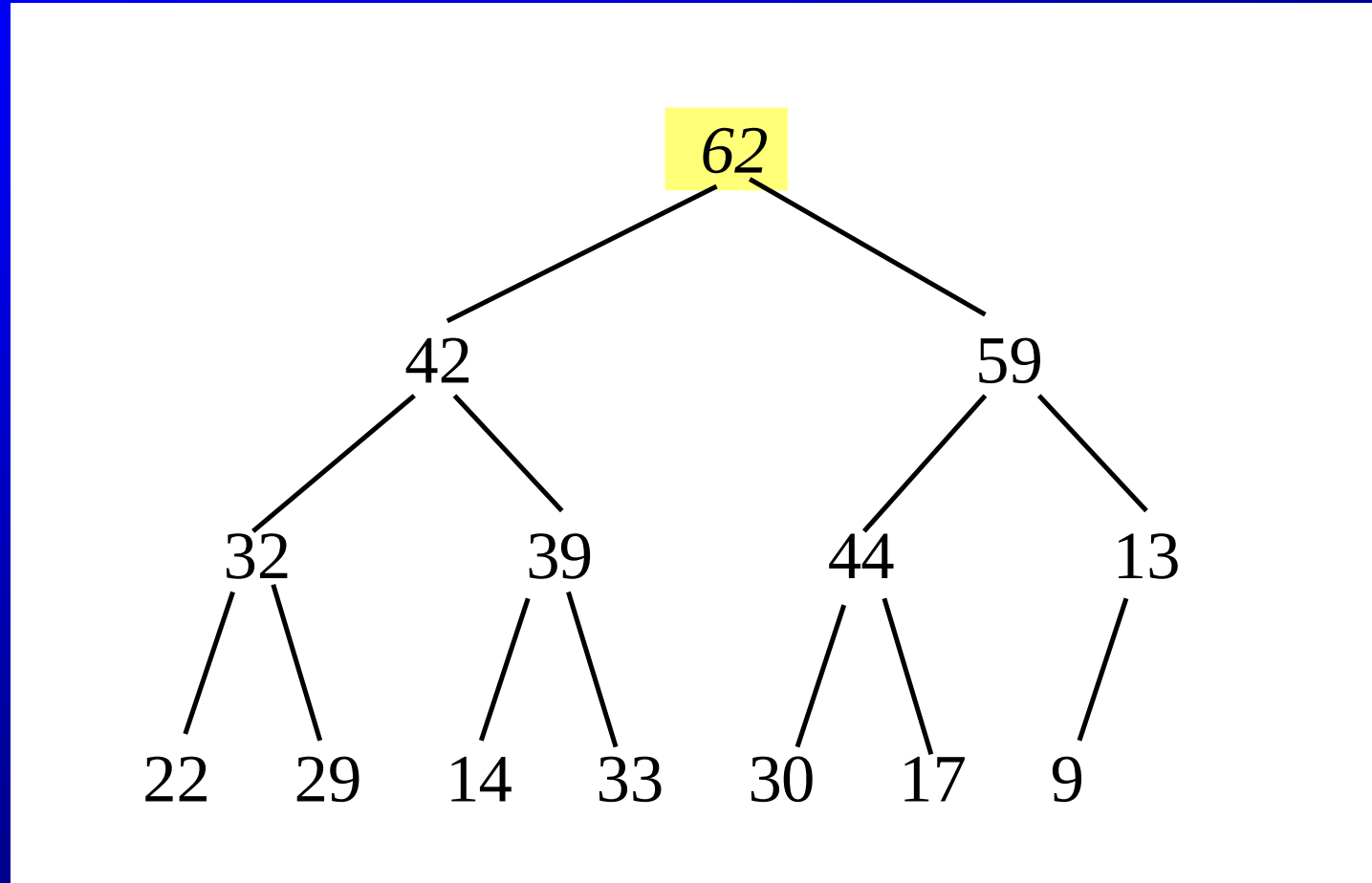


(b) After swapping 88 with 22



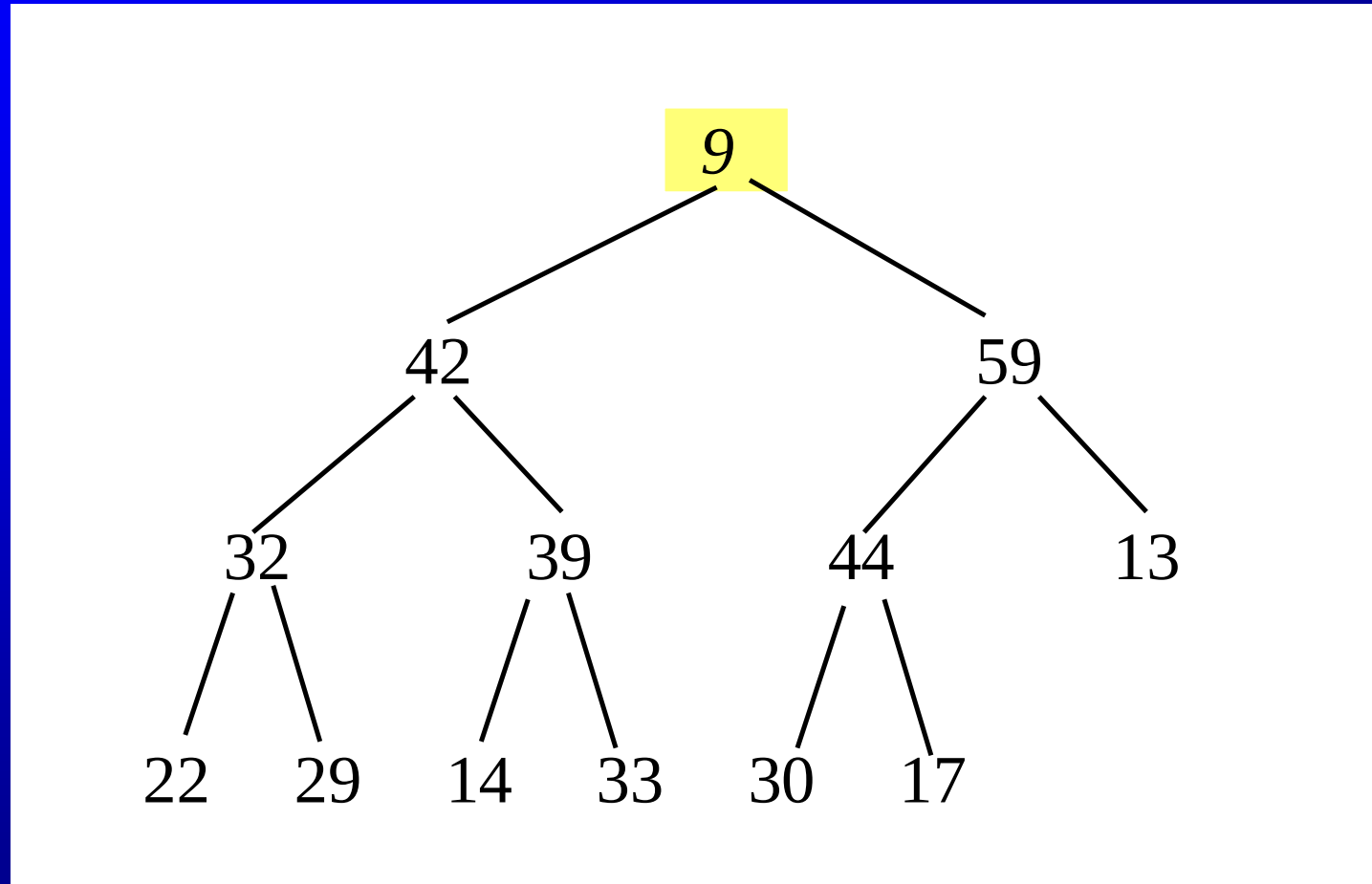
Removing the Root and Rebuild the Tree

Removing root 62 from the heap



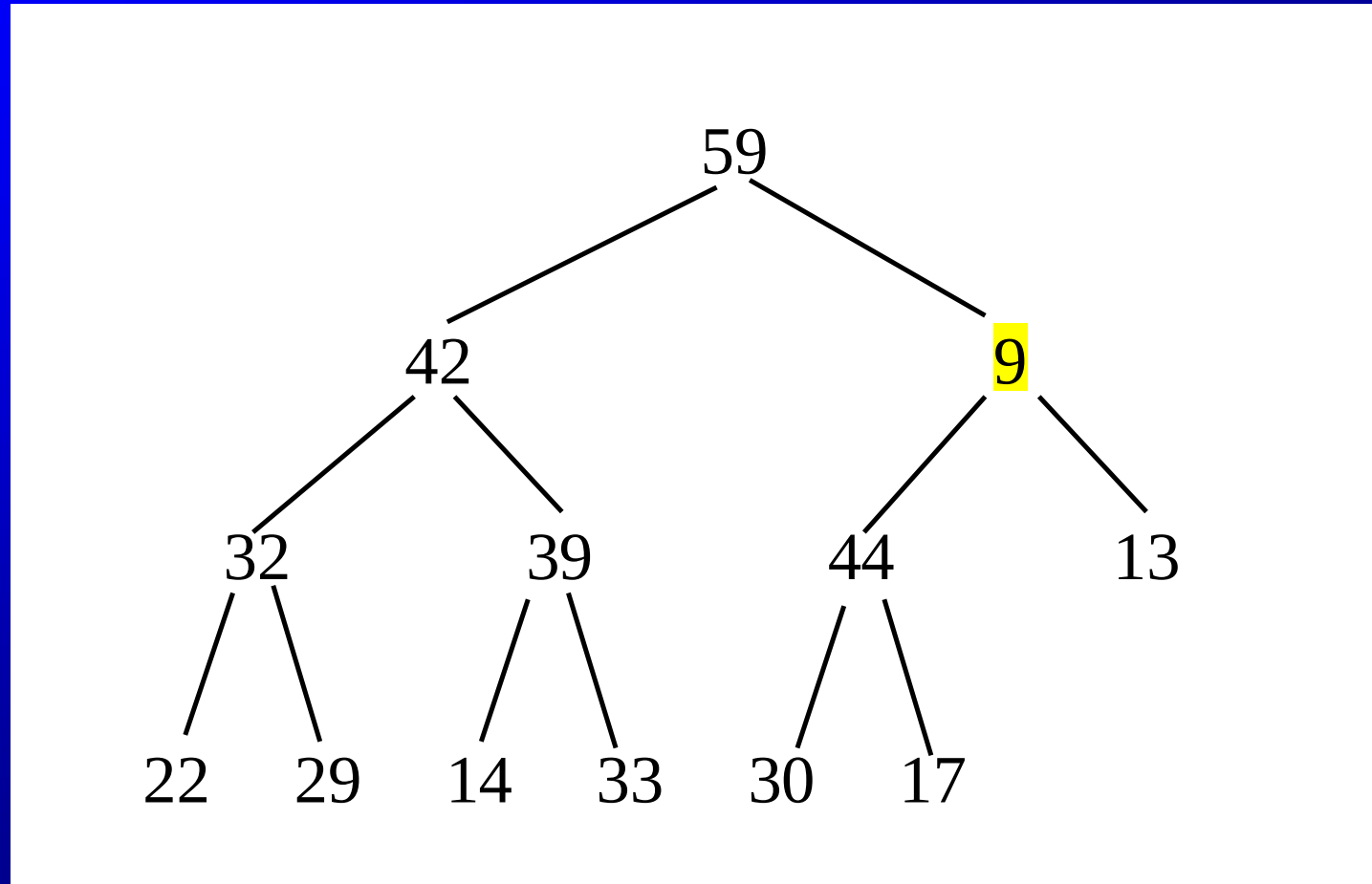
Removing the Root and Rebuild the Tree

Move 9 to root



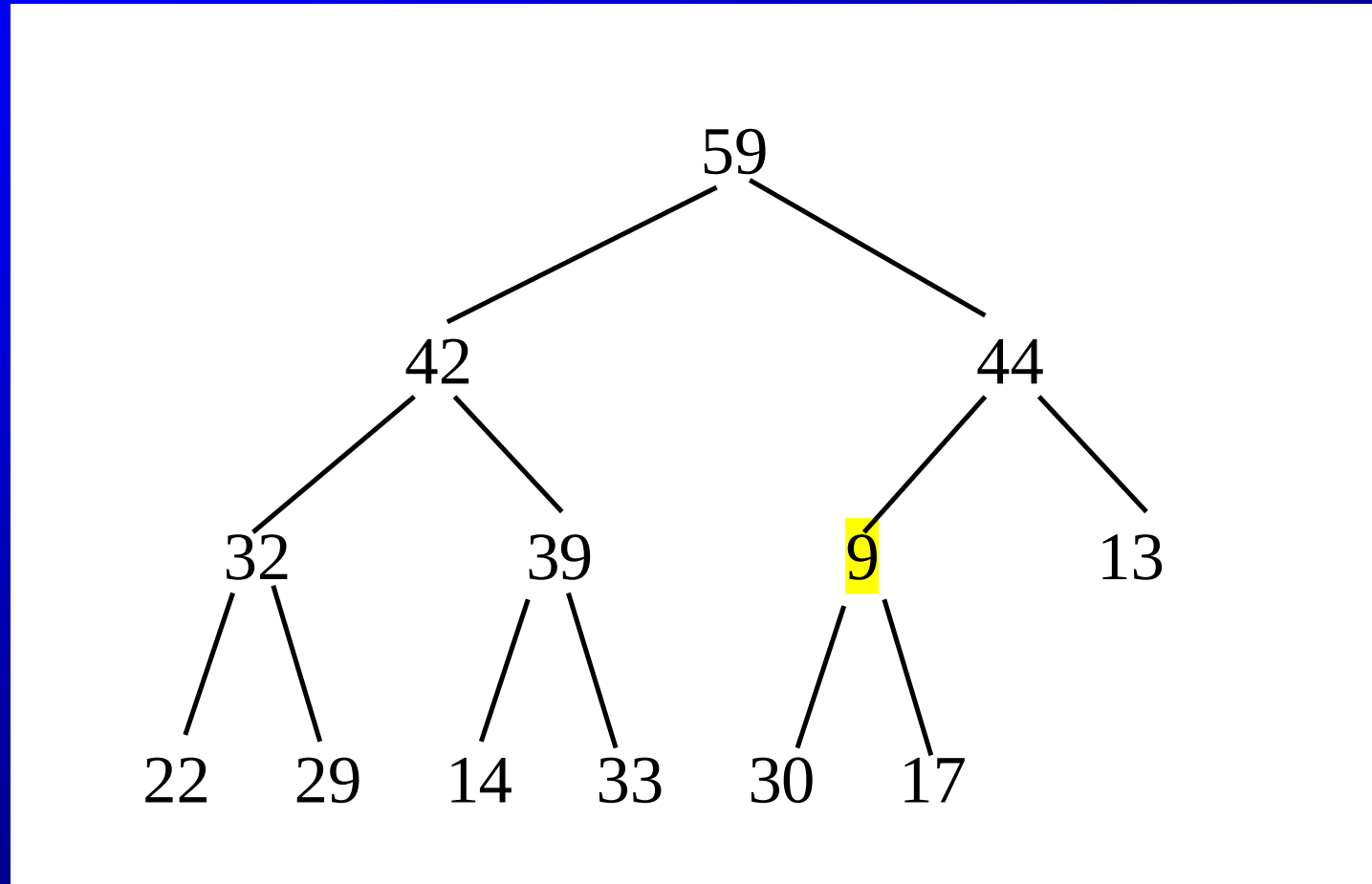
Removing the Root and Rebuild the Tree

Swap 9 with 59



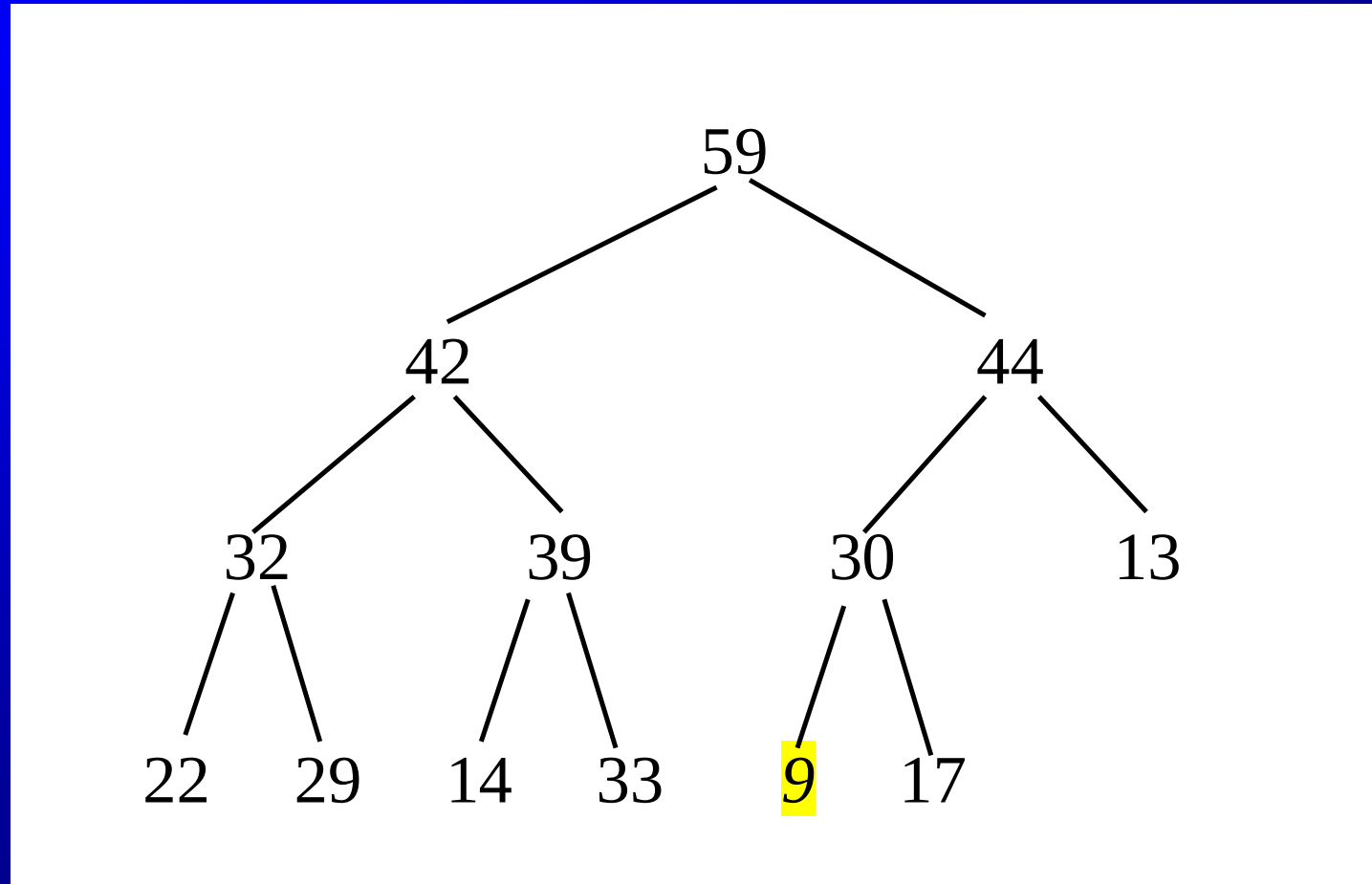
Removing the Root and Rebuild the Tree

Swap 9 with 44



Removing the Root and Rebuild the Tree

Swap 9 with 30



Heap Sort Time

Let h denote the height for a heap of n elements. Since a heap is a complete binary tree, the first level has 1 node, the second level has 2 nodes, the k th level has $2^{(k-1)}$ nodes, the $(h-1)$ th level has $2^{(h-2)}$ nodes, and the h th level has at least one node and at most $2^{(h-1)}$ nodes. Therefore,

$$1 + 2 + \dots + 2^{h-2} < n \leq 1 + 2 + \dots + 2^{h-2} + 2^{h-1}$$

$$2^{h-1} - 1 < n \leq 2^h - 1 \quad 2^{h-1} < n + 1 \leq 2^h \quad \log 2^{h-1} < \log(n + 1) \leq \log 2^h$$

$$h - 1 < \log(n + 1) \leq h \quad \log(n + 1) \leq h < \log(n + 1) + 1$$



Bucket Sort

- Bucket sort works by partitioning the elements into buckets and then return the result
- Buckets are assigned based on each element's search key
- To return the result, concatenate each bucket and return as a single array

Bucket Sort

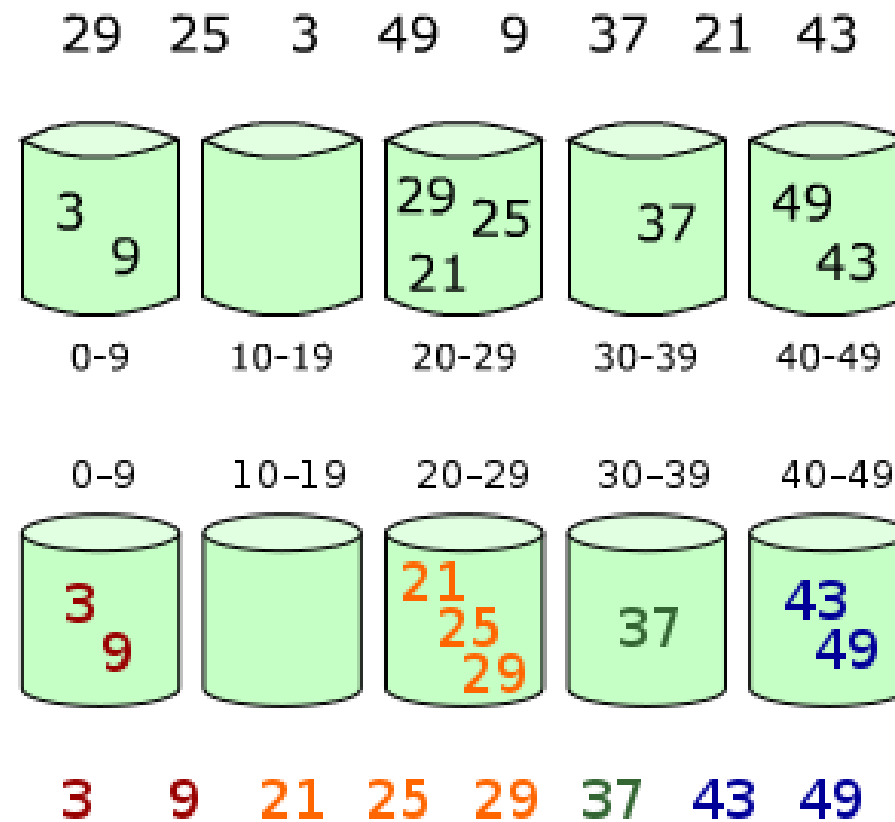
- Some variations
 - Make enough buckets so that each will only hold one element, use a count for duplicates
 - Use fewer buckets and then sort the contents of each bucket
 - Radix sort (which I'll demonstrate next)
- The more buckets you use, the faster the algorithm will run but it uses more memory

Bucket Sort

- Time complexity is reduced when the number of items per bucket is evenly distributed and as close to 1 per bucket as possible
- Buckets require extra space, so we are trading increased space consumption for a lower time complexity
- In fact Bucket Sort beats all other sorting routines in time complexity but can require a lot of space

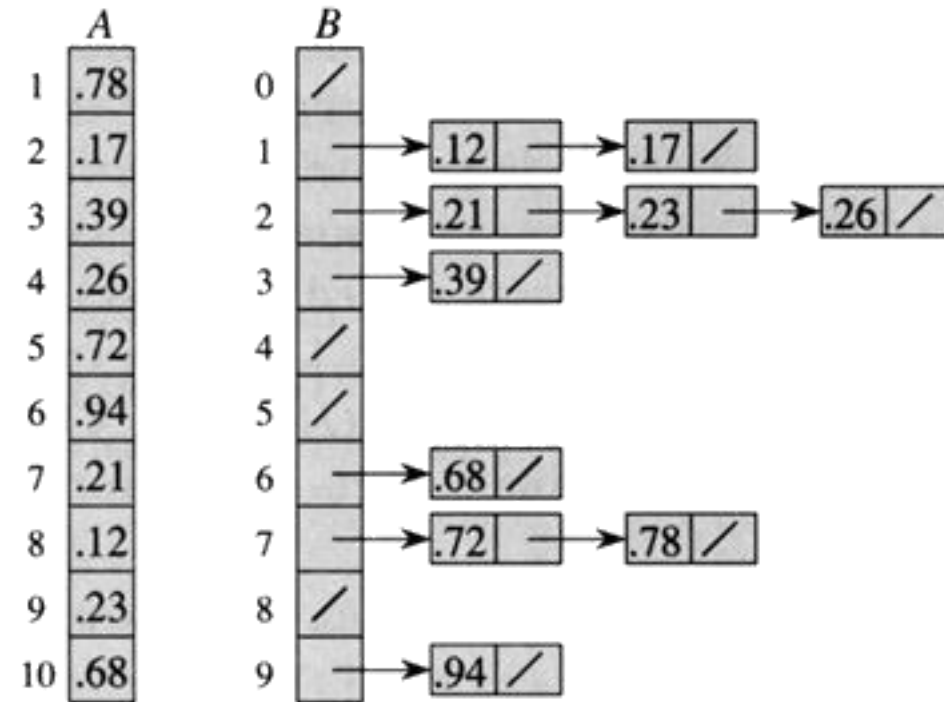
Bucket Sort

Multiple items per bucket:



Bucket Sort

In array form:



Bucket Sort Complexity

- Worst Case Complexity: $O(n^2)$
 - When there are elements of close range in the array, they are likely to be placed in the same bucket. This may result in some buckets having more number of elements than others.
 - It makes the complexity depend on the sorting algorithm used to sort the elements of the bucket.
 - The complexity becomes even worse when the elements are in reverse order. If insertion sort is used to sort elements of the bucket, then the time complexity becomes $O(n^2)$.
- Best Case Complexity: $O(n+k)$
 - It occurs when the elements are uniformly distributed in the buckets with a nearly equal number of elements in each bucket.
 - The complexity becomes even better if the elements inside the buckets are already sorted.
 - If insertion sort is used to sort elements of a bucket then the overall complexity in the best case will be linear i.e. $O(n+k)$. $O(n)$ is the complexity for making the buckets and $O(k)$ is the complexity for sorting the elements of the bucket using algorithms having linear time complexity at the best case.
- Average Case Complexity: $O(n)$
 - It occurs when the elements are distributed randomly in the array. Even if the elements are not distributed uniformly, bucket sort runs in linear time. It holds true until the sum of the squares of the bucket sizes is linear in the total number of elements.

Radix Sort

- Improves on bucket sort by reducing the number of buckets
- Maintains time complexity of $O(n)$
- Radix sort executes a bucket sort for each significant digit in the data-set
 - 100's would require 3 bucket sorts
 - 100000's would require 6 bucket sorts

Radix Sort

Sort: 36 9 0 25 1 49 64 16 81 4

First Buckets:

Bin	0	1	2	3	4	5	6	7	8	9
Content	0	1 81	-	-	64 4	25	36 16	-	-	9 49

Second Buckets:

Bin	0	1	2	3	4	5	6	7	8	9
Content	0 1 4 9	16	25	36	49	-	64	-	81	-

Radix sort

- Example:

2	0	1	0	0	1	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	1
5	1	0	1	1	0	0	0	0	1	0	0
1	0	0	1	1	1	0	1	0	1	1	0
7	1	1	1	1	0	1	0	1	0	0	0
3	0	1	1	0	0	1	1	0	1	0	1
4	1	0	0	1	1	1	1	1	1	0	0
6	1	1	0	0	1	1	0	1	1	1	1

Each sorting step must be stable.

Radix sort characteristics

- Each sorting step can be performed via bucket sort, and is thus $O(N)$.
- If the numbers are all b bits long, then there are b sorting steps.
- Hence, radix sort is $O(bN)$.

More Sorting Algorithms

- Summary – Sorting Algorithms

Algorithm	Runtime	Techniques	Usefulness
Insertionsort	$O(N^2)$	Insertion	Very small arrays
Selectionsort	$O(N^2)$	Selection	Very small arrays
Bubblesort	$O(N^2)$	Two-way passes, restricting bounds of interest	Very small arrays, mostly sorted arrays
Heapsort	$O(N \log N)$	Heaps, storing complete trees in an array	Large arrays with unknown distribution
Quicksort	$O(N \log N)$ expected, $O(N^2)$ worst case	Divide-and-conquer, swapping items into position, randomization to avoid worst-case behaviour	Large arrays without too many duplicates, parallel sorting
Mergesort	$O(N \log N)$	Divide-and-conquer, merging, external sorting	Large arrays with unknown distribution, parallel sorting
Coutingsort	$O(N+M)$	Counting	Large arrays of integers with a limited range of values
Bucketsort	$O(N + M)$	Buckets	Large arrays with reasonably uniform value distribution